

Περίληψη Java: Αφηρημένες Κλάσεις και Interfaces

Βασισμένο στις διαφάνειες 03-interfaces - αφαιρέθηκαν οι ενότητες 2, 5 και 6

Στόχος: να χρησιμοποιηθεί ως σύντομη περίληψη/υλικό προβολής για πρωτοετείς φοιτητές. Δεν περιλαμβάνει τα επεξηγηματικά κουτάκια των προηγούμενων εκδόσεων.

1. Πολυμορφισμός μεταβλητών

- Μία μεταβλητή τύπου υπερκλάσης μπορεί να δείχνει σε αντικείμενο υποκλάσης.
- Ο δηλωμένος τύπος της μεταβλητής καθορίζει τι επιτρέπεται να καλέσουμε στον κώδικα.
- Το πραγματικό αντικείμενο καθορίζει ποια υλοποίηση μεθόδου θα εκτελεστεί, όταν η μέθοδος υπάρχει και γίνεται override.

```
class Box {
    public double getWeight() { ... }
    public double calculateCost() { ... }
}
class LuxuryBox extends Box {
    public void setGiftWrap(Color c) { ... }
}
Box x = new LuxuryBox();
x.calculateCost(); // επιτρέπεται
// x.setGiftWrap(...); // δεν επιτρέπεται, γιατί η x δηλώθηκε ως Box
```

3. Γιατί χρησιμοποιούμε πολυμορφισμό

- Μας επιτρέπει να γράφουμε πιο γενικό κώδικα.
- Μία μέθοδος μπορεί να δέχεται Box και να δουλεύει με κάθε ειδικό είδος κουτιού.
- Αν αργότερα προστεθεί νέα υποκλάση, χρειάζονται λιγότερες αλλαγές στον υπόλοιπο κώδικα.

```
public void scheduleDelivery(Box x) {
    // δουλεύει με FragileBox, LuxuryBox, HeavyBox κ.λπ.
}
```

4. Πίνακες με πολυμορφισμό

- Αντί να κρατάμε ξεχωριστούς πίνακες για κάθε υποκλάση, μπορούμε να κρατάμε έναν πίνακα τύπου υπερκλάσης.
- Έτσι μπορούμε να κάνουμε κοινή επεξεργασία σε διαφορετικά αντικείμενα.
- Η σωστή calculateCost εκτελείται ανάλογα με το πραγματικό αντικείμενο κάθε θέσης.

```
Box[] boxes = new Box[300];
boxes[0] = new FragileBox();
boxes[1] = new LuxuryBox();
boxes[2] = new HeavyBox();

double totalCost = 0;
for (int i = 0; i < boxes.length; i++) {
    totalCost += boxes[i].calculateCost();
}
```

7. Αφηρημένη κλάση

- Μία αφηρημένη κλάση περιγράφει έναν γενικό τύπο, αλλά δεν θέλουμε να φτιάχνουμε απευθείας αντικείμενα από αυτή.
- Δηλώνεται με τη λέξη abstract.
- Μπορεί να έχει κανονικά πεδία, constructors και κανονικές μεθόδους.
- Μπορεί επίσης να έχει αφηρημένες μεθόδους, δηλαδή μεθόδους χωρίς σώμα.

```
abstract class DeliveryVehicle {
    private String id;

    public DeliveryVehicle(String id) {
        this.id = id;
    }

    public String getId() {
        return id;
    }

    public abstract void deliverPackage(Box b);
}
```

```
    public abstract void prepareForDelivery();  
}
```

8. Κανόνες αφηρημένων κλάσεων

- Δεν μπορούμε να κάνουμε new σε abstract class.
- Μία υποκλάση πρέπει να υλοποιήσει όλες τις abstract μεθόδους, εκτός αν είναι και αυτή abstract.
- Ο τύπος της abstract class μπορεί να χρησιμοποιηθεί κανονικά σε μεταβλητές, πίνακες και παραμέτρους.

```
// Δεν επιτρέπεται:  
// DeliveryVehicle v = new DeliveryVehicle("007");  
  
DeliveryVehicle v = new Truck("T1"); // επιτρέπεται
```

9. Interface: τι είναι

- Interface είναι ένας τύπος που περιγράφει ποιες μεθόδους πρέπει να έχει μία κλάση.
- Δεν ενδιαφέρεται για το πώς υλοποιούνται οι μέθοδοι, αλλά για το ότι υπάρχουν.
- Μια κλάση δηλώνει ότι ακολουθεί ένα interface με τη λέξη implements.
- Συνήθως χρησιμοποιούμε interface όταν θέλουμε κοινή συμπεριφορά ανάμεσα σε διαφορετικές κλάσεις.

```
interface Deliverable {  
    void deliverPackage(Box b);  
    double calculateCost();  
}
```

10. Υλοποίηση interface

- Μια κλάση που κάνει implements ένα interface πρέπει να γράψει τις μεθόδους που απαιτεί το interface.
- Αν δεν τις γράψει, η κλάση πρέπει να δηλωθεί abstract.
- Η υλοποίηση μπορεί να είναι διαφορετική σε κάθε κλάση.

```
class Truck implements Deliverable {  
    public void deliverPackage(Box b) {  
        // τρόπος παράδοσης με φορτηγό  
    }  
  
    public double calculateCost() {  
        return 5.0;  
    }  
}
```

Περίληψη Java: Είσοδος - Έξοδος και Αρχεία

Βασισμένο στις διαφάνειες 04-io - μέχρι την ενότητα 10

Στόχος: να χρησιμοποιηθεί ως σύντομη περίληψη/υλικό προβολής για πρωτοετείς φοιτητές. Δεν περιλαμβάνει τα επεξηγηματικά κουτάκια των προηγούμενων εκδόσεων.

1. Τι σημαίνει είσοδος και έξοδος

- Είσοδος είναι τα δεδομένα που διαβάζει το πρόγραμμα.
- Έξοδος είναι τα δεδομένα που γράφει ή εμφανίζει το πρόγραμμα.
- Στην Java συναντάμε συχνά `System.in`, `System.out` και `System.err`.
- Η είσοδος/έξοδος μπορεί να γίνεται από κονσόλα, αρχεία, δίκτυο ή άλλες πηγές.

2. Ορίσματα γραμμής εντολών

- Η `main` παίρνει πίνακα `String[] args`.
- Τα `args` περιέχουν τις τιμές που γράφουμε μετά το όνομα του προγράμματος.
- Χρήσιμο όταν θέλουμε να δώσουμε επιλογές ή δεδομένα χωρίς `Scanner`.

```
public class Hello {  
    public static void main(String[] args) {  
        if (args.length > 0) {  
            System.out.println("Hello, " + args[0] + "!");  
        } else {  
            System.err.println("Usage: java Hello <name>");  
        }  
    }  
}
```

3. Είσοδος από κονσόλα με Scanner

- Ο `Scanner` μπορεί να διαβάσει από το `System.in`.
- Η `nextLine()` διαβάζει ολόκληρη γραμμή.
- Η `next()` διαβάζει μία λέξη.
- Η `nextInt()` διαβάζει ακέραιο αριθμό.

```
import java.util.Scanner;  
  
public class Hello2 {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("What's your name? ");  
        String name = sc.nextLine();  
        System.out.println("Hello, " + name + "!");  
    }  
}
```

4. Η κλάση File

- Η `File` αναπαριστά ένα αρχείο ή έναν φάκελο στο σύστημα αρχείων.
- Το ότι φτιάχνουμε `File` αντικείμενο δεν σημαίνει ότι το αρχείο υπάρχει απαραίτητα.
- Μπορούμε να ελέγξουμε όνομα, απόλυτο `path`, ύπαρξη και δικαιώματα ανάγνωσης/εγγραφής.

```
File f = new File("input.txt");  
System.out.println(f.getName());  
System.out.println(f.getAbsolutePath());  
System.out.println(f.exists());  
System.out.println(f.canRead());
```

5. Σχετικά και απόλυτα paths

- Σχετικό `path`: μετρίεται από τον φάκελο από όπου τρέχει το πρόγραμμα.
- Απόλυτο `path`: δείχνει ολόκληρη τη διαδρομή στο σύστημα αρχείων.
- Σε εργασίες φοιτητών είναι πιο απλό να βάζουμε το αρχείο στον φάκελο του `project` και να χρησιμοποιούμε σχετικό `path`.

```
File f1 = new File("input.txt");  
File f2 = new File("/home/user/Documents/input.txt");  
File f3 = new File("../Documents/input.txt");
```

6. Ανάγνωση αρχείου με Scanner

- Ο Scanner μπορεί να διαβάσει από αρχείο αντί για πληκτρολόγιο.
- Η `hasNextLine()` ελέγχει αν υπάρχει επόμενη γραμμή.
- Η `nextLine()` επιστρέφει την επόμενη γραμμή.
- Στο τέλος κλείνουμε τον Scanner με `close()`.

```
File file = new File("input.txt");
Scanner sc = new Scanner(file);
while (sc.hasNextLine()) {
    String line = sc.nextLine();
    System.out.println(line);
}
sc.close();
```

7. Μέτρηση λέξεων με Scanner

- Η `hasNext()` ελέγχει αν υπάρχει επόμενη λέξη/token.
- Η `next()` διαβάζει την επόμενη λέξη.
- Με έναν μετρητή μπορούμε να μετρήσουμε πόσες λέξεις έχει ένα αρχείο.

```
Scanner sc = new Scanner(new File("book.txt"));
int count = 0;
while (sc.hasNext()) {
    sc.next();
    count++;
}
sc.close();
System.out.println("Total words: " + count);
```

8. Ανάγνωση χαρακτήρων με FileReader

- Ο `FileReader` διαβάζει χαρακτήρα-χαρακτήρα.
- Η `read()` επιστρέφει έναν ακέραιο που αντιστοιχεί στον χαρακτήρα.
- Όταν φτάσει στο τέλος του αρχείου, επιστρέφει -1.
- Είναι χρήσιμος όταν μας ενδιαφέρουν συγκεκριμένοι χαρακτήρες.

```
int c;
int count = 0;
FileReader reader = new FileReader("book.txt");
while ((c = reader.read()) != -1) {
    if (c == '!') {
        count++;
    }
}
reader.close();
```

9. Εγγραφή σε αρχείο με PrintWriter

- Ο `PrintWriter` γράφει κείμενο σε αρχείο.
- Η `println` γράφει μια γραμμή και αλλάζει γραμμή.
- Αν το αρχείο υπάρχει, συνήθως αντικαθίσταται από το νέο περιεχόμενο.
- Πρέπει να καλούμε `close()` για να αποθηκευτούν σωστά τα δεδομένα.

```
File f = new File("output.txt");
PrintWriter pw = new PrintWriter(f);
pw.println("Hello!");
pw.println("Writing is easy...");
pw.close();
```

10. Αντιγραφή αρχείου γραμμή-γραμμή

- Συνδυάζουμε Scanner για ανάγνωση και PrintWriter για εγγραφή.
- Διαβάζουμε κάθε γραμμή από το input και τη γράφουμε στο output.
- Αυτό είναι βασικό μοτίβο για πολλές ασκήσεις με αρχεία.

```
Scanner sc = new Scanner(new File("input.txt"));
PrintWriter pw = new PrintWriter("copy.txt");
while (sc.hasNextLine()) {
    pw.println(sc.nextLine());
}
sc.close();
pw.close();
```